

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра систем штучного інтелекту



Лабораторна робота № 1

з дисципліни

«Захист інформації»

Виконав:

студент групи ШІ-21

Яровой Павло Олегович

Викладач:

Фаріон М.Я.

Тема: Створення та тестування генератора псевдовипадкових чисел

Мета роботи: ознайомитись з джерелами та застосуванням випадкових чисел, алгоритмами генерування псевдовипадкових чисел та навчитись створювати програмні генератори псевдовипадкових чисел для використання в системах захисту інформації.

Варіант 12

Теоретичні відомості:

Класифікація випадкових чисел: істинні (TRNG) та псевдовипадкові (PRNG)

Істинні випадкові числа (True Random Number Generator, TRNG) отримуються з фізичних процесів, що вважаються непередбачуваними: тепловий шум, радіоактивний розпад, квантові ефекти, атмосферний шум тощо. Такі генератори використовують апаратні сенсори для збору ентропії з навколишнього середовища, після чого отримані дані можуть додатково оброблятися для підвищення рівномірності розподілу.

TRNG забезпечують справжню непередбачуваність і використовуються там, де потрібна максимальна ентропія, наприклад, для генерації криптографічних ключів найвищого рівня безпеки, у фінансових системах, сертифікованих центрах тощо. Однак такі генератори мають обмеження щодо швидкодії, вартості та складності інтеграції в програмне забезпечення.

Псевдовипадкові числа (Pseudorandom Number Generator, PRNG) - це послідовності, що генеруються детермінованими алгоритмами, ініціалізуються початковим значенням (seed) і імітують властивості випадковості. PRNG широко використовуються в чисельних симуляціях, статистиці, комп'ютерній графіці, іграх, машинному навчанні та багатьох інших сферах, де потрібна велика кількість випадкових чисел із високою швидкістю.

PRNG мають такі властивості:

1. **Детермінованість:** однакове початкове зерно завжди породжує ту саму послідовність.
2. **Відтворюваність:** важлива для тестування, відлагодження, відтворюваних експериментів.

3. **Періодичність:** послідовність має певний період, після якого починає повторюватися.
4. **Швидкодія:** значно перевищує TRNG.

Недоліки PRNG полягають у потенційній передбачуваності та обмеженій ентропії, що робить їх непридатними для криптографічних задач без додаткових заходів безпеки.

Лінійний конгруентний генератор (LCG): формула, параметри, приклади

3.1. Формула LCG

Лінійний конгруентний генератор задається рекурентним співвідношенням:

$$x_{N+1} = (a * x_n + c) \bmod m \text{ де:}$$

1. (x_n) - поточне значення,
2. (a) - множник,
3. (c) - приріст,
4. (m) - модуль порівняння,
5. (x_0) - початкове значення (зерно, seed).

Варіанти LCG:

1. Мультиплікативний конгруентний генератор (MCG, Lehmer RNG): $(c = 0)$.
2. Змішаний конгруентний генератор: $(c \neq 0)$.

3.2. Вибір параметрів та період

Якість LCG критично залежить від вибору параметрів (a, c, m) . Для досягнення максимального періоду (довжина циклу $= (m)$), параметри повинні задовольняти умови теореми Халла–Добелла:

1. (c) та (m) — взаємно прості.
2. $(a-1)$ кратно всім простим дільникам (m) .
3. Якщо (m) кратно 4, то $(a-1)$ також кратно 4.

Критерії якості PRNG (статистичні та практичні)

Статистичні критерії

1. **Рівномірність розподілу:** всі значення в діапазоні повинні з'являтися з однаковою ймовірністю.
2. **Незалежність:** відсутність кореляцій між послідовними значеннями.

3. **Відсутність коротких циклів:** період повинен бути максимально довгим.
4. **Відсутність багатовимірних кореляцій:** у n -вимірному просторі точки повинні рівномірно заповнювати простір, а не лежати на гіперплощинах.
5. **Висока ентропія:** кожен біт послідовності має бути максимально непередбачуваним.
6. **k-розподіленість:** кожна можлива комбінація k -бітів з'являється однаково часто протягом періоду (окрім комбінації з усіх нулів, яка з'являється на одну менше).

Практичні критерії

1. **Великий період:** для сучасних задач бажано (2^{64}).
2. **Висока швидкодія:** важливо для симуляцій, ігор, ML.
3. **Простота реалізації:** мінімальна кількість операцій.
4. **Мала потреба в пам'яті:** особливо для вбудованих систем.
5. **Портативність:** можливість використання на різних платформах.
6. **Відновлення після поганого стану:** генератор не повинен "зависати" у поганих підпросторах стану.
7. **Відтворюваність:** однакове зерно — однакова послідовність.
8. **Відсутність передбачуваності:** особливо важливо для криптографії.

Тестування випадковості: NIST SP 800-22

NIST SP 800-22 — це набір статистичних тестів, розроблений Національним інститутом стандартів і технологій США для оцінки випадковості бітових послідовностей, зокрема для криптографічних застосувань. Він містить 15 основних тестів, кожен з яких перевіряє певний аспект випадковості.

Основні тести NIST SP 800-22

1. **Частотний (монобітний) тест:** перевіряє, чи кількість нулів та одиниць приблизно однакова.
2. **Частотний тест по блоках:** оцінює рівномірність розподілу одиниць у блоках.
3. **Тест серій:** перевіряє кількість серій однакових бітів.
4. **Тест найдовшої серії з одиниць:** оцінює, чи довжина найдовшої серії одиниць відповідає очікуванню.
5. **Тест рангу бінарних матриць:** виявляє лінійну залежність у підрядках.

6. **Тест на основі дискретного перетворення Фур'є:** виявляє періодичність у послідовності.
7. **Non-overlapping/Overlapping Template Matching:** шукає задані шаблони.
8. **Universal Statistical Test:** оцінює ступінь стиснення послідовності.
Approximate Entropy Test: оцінює ентропію підрядків.
9. **Random Excursions Test:** аналізує поведінку випадкових блукань.
10. **Serial Test, Linear Complexity Test:** оцінюють складність та серійну структуру.

Програмна реалізація

```
1  import tkinter as tk
2  from tkinter import filedialog, messagebox
3  import math
4  import random
5
6  M = 2**21 - 1
7  A = 8**3
8  C = 144
9  SEED = 3
10
11  numbers = []
12
13
14  def lcg(n, seed=SEED, a=A, c=C, m=M): 1 usage
15      nums = []
16      x = seed
17      for _ in range(n):
18          x = (a * x + c) % m
19          nums.append(x)
20      return nums
21
22
23  def gcd(a, b): 1 usage
24      while b:
25          a, b = b, a % b
26      return a
27
28
29  def find_period(seed=SEED, a=A, c=C, m=M): 1 usage
30      seen = set()
31      x = seed
32      period = 0
```

```
34     while x not in seen:
35         seen.add(x)
36         x = (a * x + c) % m
37         period += 1
38
39     return period
40
41
42 def cesaro_test(nums): 2 usages
43     coprime = 0
44     total = 0
45
46     for i in range(0, len(nums) - 1, 2):
47         if gcd(nums[i], nums[i + 1]) == 1:
48             coprime += 1
49         total += 1
50
51     if total == 0 or coprime == 0:
52         return None
53     p = coprime / total
54     return math.sqrt(6 / p)
55
56
57 def system_test(n): 1 usage
58     nums = [random.randint(a: 1, M) for _ in range(n)]
59     return cesaro_test(nums)
60
61
62 def generate(): 1 usage
63     global numbers
64     try:
```

```

65         n = int(entry.get())
66     except:
67         messagebox.showerror( title: "Помилка", message: "Введіть число")
68         return
69     numbers = lcg(n)
70     text.delete( index1: 1.0, tk.END)
71     for num in numbers:
72         text.insert(tk.END, str(num) + "\n")
73
74     def save(): 1 usage
75     if not numbers:
76         messagebox.showwarning( title: "Увага", message: "Немає даних")
77         return
78
79     file = filedialog.asksaveasfilename(defaultextension=".txt")
80
81     if file:
82         with open(file, "w") as f:
83             for num in numbers:
84                 f.write(str(num) + "\n")
85
86
87     def show_period(): 1 usage
88         period = find_period()
89         messagebox.showinfo( title: "Період", message: f"Період генератора: {period}")
90
91
92     def test_cesaro(): 1 usage
93     if not numbers:
94         messagebox.showwarning( title: "Увага", message: "Спочатку згенеруйте числа")

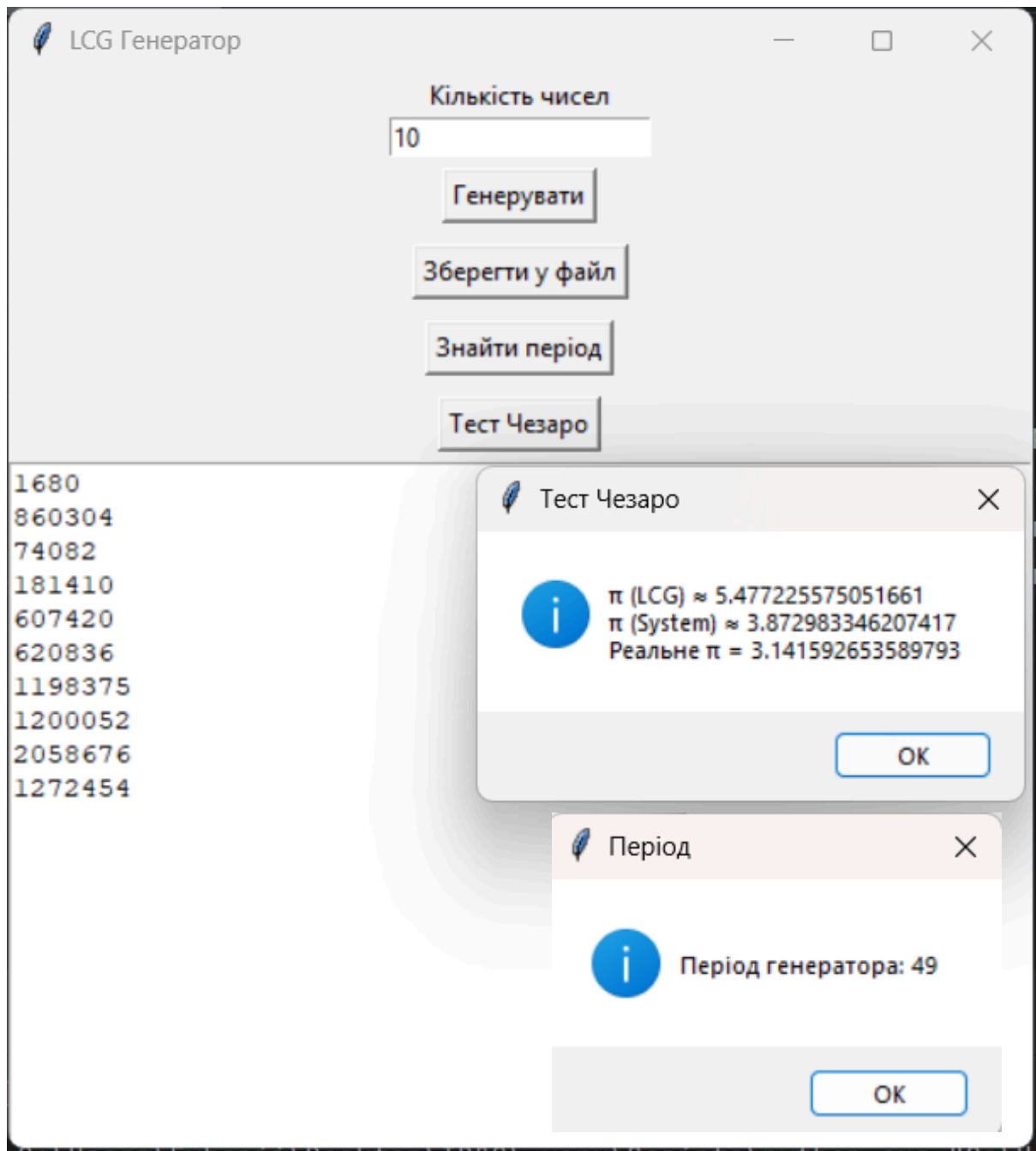
```

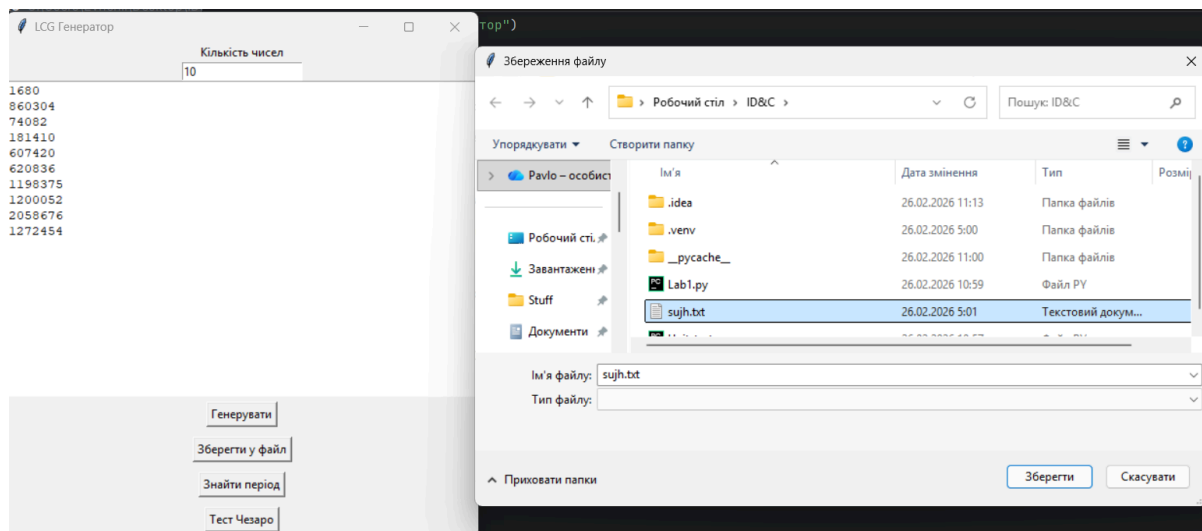
```

92 def test_cesaro(): 1 usage
93     if not numbers:
94         messagebox.showwarning(title="Увага", message="Спочатку згенеруйте числа")
95         return
96
97     pi_lcg = cesaro_test(numbers)
98     pi_sys = system_test(len(numbers))
99
100     msg = (
101         f"π (LCG) ≈ {pi_lcg}\n"
102         f"π (System) ≈ {pi_sys}\n"
103         f"Реальне π = {math.pi}"
104     )
105     messagebox.showinfo(title="Тест Чезаро", msg)
106
107 root = tk.Tk()
108 root.title("LCG Генератор")
109
110 tk.Label(root, text="Кількість чисел").pack()
111
112 entry = tk.Entry(root)
113 entry.pack()
114 entry.insert(index=0, string="1000")
115
116 tk.Button(root, text="Генерувати", command=generate).pack(pady=5)
117 tk.Button(root, text="Зберегти у файл", command=save).pack(pady=5)
118 tk.Button(root, text="Знайти період", command=show_period).pack(pady=5)
119 tk.Button(root, text="Тест Чезаро", command=test_cesaro).pack(pady=5)
120 text = tk.Text(root, height=20, width=60)
121 text.pack()
122 root.mainloop()

```


Хід роботи:





Висновок:

Аналіз результату: короткий період вказує на сильну циклічність і погану статистичну якість; це пояснює значне відхилення в оцінці π . Невелика вибірка також підвищує дисперсію оцінок для обох генераторів.

Генерація псевдовипадкових чисел — це не лише питання швидкодії чи простоти реалізації, а й питання якості, безпеки та відповідності задачі. LCG — це класика, яка залишається актуальною для простих задач, але для сучасних вимог, особливо у сфері безпеки, слід використовувати перевірені сучасні генератори та дотримуватися стандартів тестування.